

Mini-Projet B : Intégration numérique

MGA802 — Introduction à la programmation avec Python

Loïc Jacob, Fabien Koch, Guillaume Pissang

9 juin 2026

1 Introduction et objectifs

L'objectif de ce projet est de calculer numériquement l'aire sous la courbe d'une fonction polynomiale du troisième ordre, puis de comparer plusieurs méthodes d'intégration selon deux critères : la **précision** (l'erreur par rapport à la solution exacte) et le **temps de calcul**. On cherche en particulier à démontrer l'intérêt de la bibliothèque NumPy pour le calcul numérique.

La fonction étudiée est :

$$f(x) = p_1 + p_2 x + p_3 x^2 + p_4 x^3 \quad (1)$$

et l'on souhaite évaluer l'intégrale $I = \int_a^b f(x) dx$.

Comme f est un polynôme, l'intégrale possède une solution analytique exacte obtenue par la primitive :

$$I_{\text{exact}} = \left[p_1 x + \frac{p_2}{2} x^2 + \frac{p_3}{3} x^3 + \frac{p_4}{4} x^4 \right]_a^b \quad (2)$$

Cette valeur exacte sert de référence : l'**erreur** d'une méthode numérique est définie comme la valeur absolue de l'écart entre son résultat et I_{exact} .

2 Méthodes d'intégration

Chaque méthode découpe l'intervalle $[a, b]$ en n segments réguliers de largeur $h = (b - a)/n$. Trois méthodes sont implémentées, chacune en deux versions : une en Python de base (avec une boucle `for`) et une vectorisée avec NumPy.

Méthode des rectangles (point milieu). La fonction est évaluée au centre de chaque segment, formant un rectangle :

$$I_{\text{rect}} = h \sum_{i=0}^{n-1} f\left(a + \left(i + \frac{1}{2}\right) h\right) \quad (3)$$

Méthode des trapèzes. Sur chaque segment $[x_i, x_{i+1}]$, l'aire est approchée par un trapèze :

$$I_{\text{trap}} = \sum_{i=0}^{n-1} (x_{i+1} - x_i) \frac{f(x_i) + f(x_{i+1})}{2} \quad (4)$$

Méthode de Simpson. Sur chaque segment, la fonction est approchée par une parabole :

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right] \quad (5)$$

Cette méthode est **exacte** pour les polynômes de degré inférieur ou égal à 3.

Ces implémentations sont enfin comparées aux méthodes pré-programmées de SciPy (`scipy.integrate.trapezoid` et `scipy.integrate.simpson`).

Cas étudiés. Deux jeux de coefficients ont été testés pour vérifier que les conclusions ne dépendent pas d'un cas particulier :

- **Cas standard** : $f(x) = 1 + 2x + 3x^2 + 4x^3$ sur $[0, 2]$, d'aire exacte $I_{\text{exact}} = 30$.
- **Cas à coefficients négatifs** : $f(x) = 2 - 3x + x^2 - x^3$ sur $[-1, 3]$, d'aire exacte $I_{\text{exact}} \approx -14,667$.

3 Résultats numériques pour $n = 100$

Le tableau 1 présente les aires et erreurs obtenues pour $n = 100$ segments, pour les deux cas étudiés.

Méthode		Erreur (cas standard)	Erreur (cas négatif)
Rectangles	Python / NumPy	$1,00 \times 10^{-3}$	$1,07 \times 10^{-3}$
Trapèzes	Python / NumPy	$2,00 \times 10^{-3}$	$2,13 \times 10^{-3}$
Simpson	Python / NumPy	0 (machine)	$3,55 \times 10^{-15}$
Trapèzes	SciPy	$2,00 \times 10^{-3}$	$2,13 \times 10^{-3}$
Simpson	SciPy	$3,55 \times 10^{-15}$	$1,78 \times 10^{-15}$

TABLE 1 – Erreur de chaque méthode pour $n = 100$ segments.

On constate dès cette valeur de n que la méthode des trapèzes a une erreur environ deux fois supérieure à celle des rectangles, tandis que la méthode de Simpson atteint déjà la précision machine (de l'ordre de 10^{-15}). Les versions Python et NumPy donnent exactement le même résultat numérique, ce qui est attendu puisqu'elles effectuent le même calcul mathématique ; seule leur vitesse diffère (section 5). Les méthodes SciPy fournissent des résultats identiques aux nôtres, ce qui valide nos implémentations.

4 Analyse de la convergence (précision)

La convergence est étudiée en faisant croître le nombre de segments n (de 10 à environ 8×10^4) et en observant l'évolution de l'erreur. Les graphiques sont tracés en échelle logarithmique sur les deux axes : sur ce type de graphique, une méthode d'ordre constant apparaît comme une droite, dont la pente indique la vitesse de convergence.

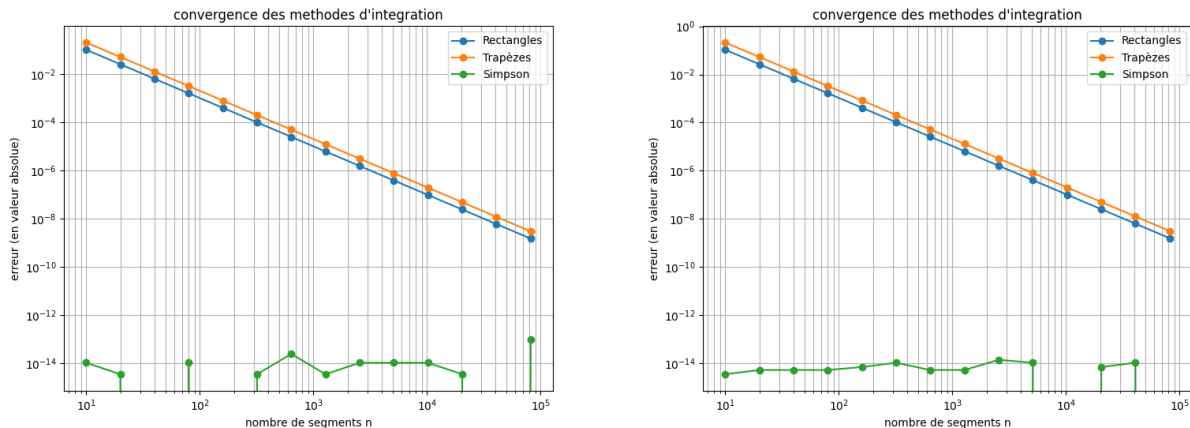


FIGURE 1 – Convergence de l'erreur en fonction du nombre de segments n . À gauche le cas standard, à droite le cas à coefficients négatifs.

Interprétation. Les deux graphiques montrent le même comportement, ce qui confirme la robustesse des conclusions :

- Les méthodes des **rectangles** et des **trapèzes** forment deux droites décroissantes de même pente : leur erreur est divisée par environ 4 chaque fois que n double, ce qui correspond à une convergence d'ordre 2.
- L'erreur des trapèzes est légèrement supérieure à celle des rectangles (point milieu), les deux courbes restant néanmoins très proches.
- La méthode de **Simpson** reste plate au niveau du plancher machine (de l'ordre de 10^{-14}), quelle que soit la valeur de n . Cela confirme qu'elle est exacte pour un polynôme de degré 3 : l'erreur

résiduelle ne provient que des arrondis de l'ordinateur, et non de la méthode. Les pics verticaux visibles sur les courbes de Simpson correspondent à des valeurs où l'erreur est exactement nulle, ce qui ne peut pas être représenté en échelle logarithmique.

5 Analyse du temps de calcul

Le temps d'exécution de chaque méthode a été mesuré pour les mêmes valeurs de n à l'aide de `perf_counter`, et la bibliothèque `timeit` a été utilisée pour obtenir des moyennes plus stables sur 1000 répétitions. Le tableau 2 présente les temps moyens (`timeit`) pour $n = 100$, cas standard.

Méthode	Python	NumPy
Rectangles	23 μs	11 μs
Trapèzes	34 μs	10 μs
Simpson	17 μs	8 μs

TABLE 2 – Temps moyen par appel (`timeit`, 1000 répétitions, $n = 100$, cas standard).

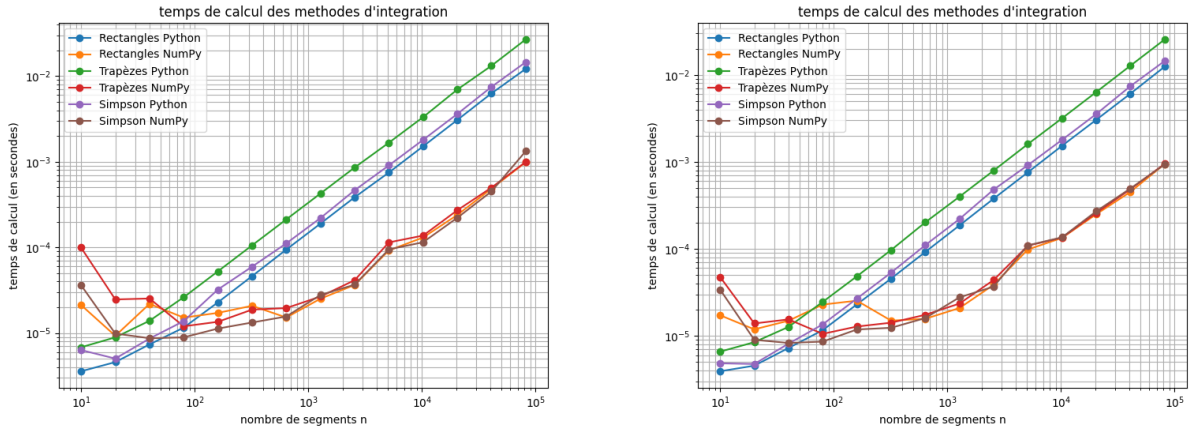


FIGURE 2 – Temps de calcul en fonction du nombre de segments n , pour les versions Python de base et NumPy de chaque méthode.

Interprétation.

- Pour de **petits** n , toutes les versions sont comparables, voire NumPy peut être légèrement plus lente : le coût de préparation des tableaux n'est pas encore rentabilisé.
- Pour de **grands** n , les versions Python de base (boucle `for`) deviennent nettement plus lentes : leur temps croît plus vite que celui des versions NumPy, et l'écart entre les deux familles de courbes se creuse.
- Les mesures `timeit` confirment ce gain : à $n = 100$, NumPy est déjà environ 2 à 3 fois plus rapide que Python de base (par exemple 34 μs contre 10 μs pour les trapèzes). Cet écart s'amplifie pour les grands n .
- Cela illustre l'intérêt de la **vectorisation** : remplacer une boucle Python par des opérations appliquées d'un coup sur des tableaux entiers.

6 Conclusion

Cette étude a permis de comparer trois méthodes d'intégration numérique selon leur précision et leur temps de calcul, sur deux jeux de coefficients menant aux mêmes conclusions.

Sur le plan de la **précision**, la méthode de Simpson est de loin la plus performante pour le polynôme étudié : elle atteint la précision machine indépendamment de n , car elle est exacte pour les polynômes de degré au plus 3. Les méthodes des rectangles et des trapèzes convergent toutes deux à l'ordre 2, l'erreur diminuant régulièrement avec l'augmentation du nombre de segments.

Sur le plan du **temps de calcul**, la vectorisation avec NumPy apporte un gain important dès que le nombre de segments devient grand, là où les boucles Python de base montrent leurs limites. Les méthodes pré-programmées de SciPy offrent une alternative fiable et rapide, sans avoir à réécrire l'algorithme, et ont servi à valider nos propres implémentations.

En résumé, le choix d'une méthode résulte d'un compromis entre la précision recherchée, le coût de calcul et la simplicité d'implémentation. Pour ce problème, la méthode de Simpson associée à une implémentation vectorisée constitue le meilleur compromis. Il convient toutefois de noter que les temps mesurés dépendent de la machine utilisée ; c'est le rapport entre les versions Python et NumPy, et non la valeur absolue en secondes, qui constitue le résultat significatif.